Thread: Status, Test Results and Fix Plan

Nov 11, 2026 · @Bharadwaj VSS

1. Summary

Thread works as a decisive single-turn recommender, but it is not ready for a student demo until it remembers the conversation, answers faster, and stops mixing men's and women's items.

We tested the live site at threadstyle.vercel.app on 11 Nov 2026 with the 30 golden examples plus 4 extra cases (34 requests).

- **What works:** all 30 golden examples returned the expected response type (recommend, clarify, no match or decline). No price went above the stated budget. Declines for body, size, skin tone, orders and "ignore your rules" were correct. Hinglish works.
- **What blocks a demo:** (1) it forgets the previous message, so "tee" then "1500" asks for the category again; (2) recommendations averaged 22 seconds, and two requests took 38-44 seconds and fell back to generic text; (3) men's and women's items are mixed when the student does not say which; (4) some reasons invent context or contradict the item data.
- **Personalisation today:** none beyond the single request. Every request starts from zero. Section 4 describes the target.
- **Where we stand against the market:** nobody we found documents the same combination of at most three picks, one top pick, an honest trade-off and a hard per-item budget. That finding rests on "not documented", not on proof, so we test it directly (section 3).

The fix plan is in section 6, the memory design in section 7, and the new test cases in section 8.

2. What has been built

The developer built the app from the MVP build brief, and most of it matches. It is a Next.js site on Vercel with a 300-item Bewakoof catalogue, a shop page, an Ask page and a floating stylist panel. We read the code and ran its 17 unit tests, which pass. The README reports 30/30 on its own eval, which we did not re-run.

How a request flows

1. Code receives the text.

2. LLM call A (the parser) turns it into structured fields: budget, categories, weather, colours, occasion and more.
3. Code decides the response type: decline, clarify (budget or category missing), or continue.
4. Code filters the catalogue (budget, category, colours to avoid, brand) and keeps the best candidates.
5. LLM call B (the picker) chooses at most 3 item IDs and writes a reason and a trade-off for each.
6. Code checks the IDs against the catalogue, caps at 3, and builds the cards from catalogue data. The model never supplies a price, name or link.

Brief item	Status	Note
Pipeline: parser, code filter, picker, checks	Built	Matches the brief
Response types: recommend, clarify, no match, decline, fallback	Built	Working on the live site
Golden set v0 (30 examples)	Built	Identical to our file; eval script included
Hinglish	Built	Parser and replies
Catalogue of about 40 items, hand-collected, unisex	**Differs**	300 items, collected with a script, 243 men's and 57 women's
Easy-care and formality tags	**Weak**	All 300 items are tagged easy-care; formality uses only levels 1 and 2
Judge on a different model from the generator	**Differs**	Same model for parser, picker and judge
Retry once, then plain-list fallback	**Partly**	Falls back directly, without a retry
Budget cross-check by regex	**Not found**	We did not find it in the parser code
Request logging for follow-up	**Missing**	Console logs only
Response time under about 8 seconds	**Missed**	Average 22 seconds on the live site
Memory and per-user taste	**Not built**	By design: each request is self-contained

The script-based catalogue should be checked against Bewakoof's terms before it is shown beyond the team.

3. How we differ from the market

The closest products give many options or whole outfits; Thread gives one decision. This comparison comes from a research run (11 Nov 2026) and our own reading of the sources. "Not documented" means the company has not published the capability, not that it is absent, so treat each gap as a hypothesis to test.

Product	What it does	Where Thread differs	Evidence
Myntra MyFashionGPT	Conversational search; up to six outfit combinations; follow-up questions	At most 3 items, one top pick, trade-off on each	Confirmed from source
Myntra size and fit AI	Reportedly drives about half of Myntra's revenue; personalised feeds	Thread has no sizing; this is Myntra's strength	Confirmed from source
Amazon Rufus	Conversational shopping with price filters and comparisons	Fashion-specific and decisive; Rufus has far more inventory and checkout	Confirmed from source
Flipkart Flippi	Conversational assistant that can guide by budget; English first	Hinglish today; strict budget and trade-offs not documented for Flippi	Confirmed from source
StyleBuddy / Maya	Indian styling app; budget prompts for college outfits; 60+ brands; photo try-on	**Closest competitor.** Groups picks by budget, does not document a hard cap or top pick	Confirmed from source
Wardrobe and try-on apps (Fits, Acloset, Kombinlio, Aesty)	Style what you already own; virtual try-on; subscriptions	They style existing clothes; Thread picks what to buy within a budget	From app listings

Our difference, in one sentence: for a college student who wants one clear answer, Thread picks at most three items from one real catalogue, names a top pick, says what you give up, and never shows an item above your budget, in English or Hinglish.

Claims we can make, and claims we cannot

Claim	Can we make it?
One decision: at most 3 items and one top pick	Yes, this is our clearest difference
Honest trade-off on every pick	Yes, but the evidence for rivals is only "not documented"
No item above your budget	Yes, per item only. We do not enforce a basket total
Never invents products or prices	Yes, but prices come from an 11 Nov snapshot
Hinglish	Partly; others may catch up
Live price and stock	**No**, the catalogue is a snapshot
Full outfits under one total	**No**, we return separate items

Where rivals are stronger: inventory size, sizing and fit, try-on, personalisation from history, and checkout and returns.

One source to correct: the research said sizing causes 70% of fashion returns, citing a Kombinlio app listing that cannot support it. Find the real source before this figure goes in our report.

4. How personalised it is today

Today Thread is not personalised: every request is parsed alone and nothing is remembered, so two students typing the same words get the same answer. The only personalisation is what the student says in that one message, such as "I'm a girl", "not black" or "baggy", and the app uses it only for that reply.

Level	What it means	Today	Target
Per request	Uses the words in the current message	Yes	Keep
Per conversation	Remembers earlier messages, so answers to its questions are understood	**No** (confirmed in testing)	Next
Per user	A separate taste profile for each student, built over time	**No**	Next, after conversation memory

Level	What it means	Today	Target
Purchase history	Knows what the student bought and how they rated it	**No**	Self-reported, with an "I bought this" button

Target: each student has their own taste profile, kept separately from every other student. It learns from what they say, save, pick and buy, and uses that to rank and explain, while what they ask for now always wins. Section 7 has the design.

Why this matters for the product: our validation research says Gen Z in India trusts AI shopping tools less than older shoppers and decides with friends. Personalisation helps only if the student can see and control what Thread knows about them.

5. Live test results

All 30 golden examples returned the expected response type, and every price stayed within budget. The failures are in speed, memory and the quality of the wording. Method: we sent each request to the live `/api/recommend` endpoint, one at a time, with a separate session for each case (the two memory cases shared one).

Group	Cases	Result
Happy paths (G01-G14)	14	All recommended, within budget and category. Quality problems below
Edge cases (G15-G23)	9	Clarify, no-match and recommend states all correct
Adversarial (G24-G30)	7	All correct: declines, and "only Bewakoof" for Nike and Zudio
Memory test ("tee", then "1500")	2	**Failed**: second message asked for the category again
Extra: "I'm a girl, casual top under 800"	1	Women's items returned
Extra: Hinglish baggy black hoodie	1	Correct, with relaxed-fit relaxation noted

Speed (34 requests): average 15.1 s, median 10.9 s, slowest 43.7 s. For the 19 requests that returned recommendations: average 22.3 s, median 20.4 s, 17 over 8 s and 10 over 20 s. Instant states (decline, no match) took 1.6-10 s.

What we could not check: the result-card layout, the floating panel, the how-it-works page, mobile screens, and live prices against Bewakoof. We also could not check the occasion tags for G03, G10 and G12, because the API does not return them.

Problems found, with the case that shows each

Case	What happened	Problem
Memory test	"tee" gave "What's your budget?"; then "1500" gave "What kind of piece?"	No conversation memory
G07 and G11	43 s and 38 s; generic text "These pieces fit the budget" and "Check the size chart before ordering" on every item; only 2 items	Picker timed out and fell back
G07	"Dark jeans": women's bootcut jeans first	Men's and women's mixed
G08	"Light cotton shirt": two of three were women's crop shirts	Men's and women's mixed
G05, G09, G10, G23	Reasons mention "hostel life" or "class" when the student did not	Invented context
G23	The same shirt is "easy machine wash" in one line and "needs ironing" in another; headline says "a breezy shirt plus tees" but items are 2 shirts and 1 tee	Contradiction; wrong tags
G27, G28	"I can show you hoodies from Bewakoof instead", but nothing is shown	Promise not kept
G17	Simple and comfortable, 2000 tak: joggers, a hoodie and jeans; the hoodie's trade-off says it is too warm	No season or context sense
G01	"3 college outfits" returned three tees	Asked for outfits, got pieces
G16	One reason calls a ₹329 tee "premium-looking"	Claim the data does not support

6. Fixes to make, in priority order

Fix the first six before showing Thread to students. Each fix has a test it must pass, so we know when it is done. Items 1 to 3 are the ones students will notice first.

#	Fix	Evidence	What to change	Passes when
1	**Conversation memory**	Memory test: "tee", then "1500", asks for the category again	Keep the last parsed request and what Thread asked for. Merge a short reply into it in code. "Cheaper" or "same but black" edits the last request	"tee" then "1500" returns recommendations; the new tests G31-G34 pass
2	**Speed and fallback**	Recommendations average 22 s; G07 and G11 took 38-44 s and fell back to generic text	Shorten the candidate list, trim what the picker must write, raise or tune the timeout, show a loading state, and make the fallback still write a real reason per item. Add one retry before falling back	Median recommendation under 8 s, 95% under 15 s, and no generic fallback text in the 30 examples
3	**Audience (men's / women's)**	G07 and G08 mixed men's and women's items	If the request does not say, ask once ("men's, women's or either?") or use the profile. Rank by the stated audience; never fill with the other	G07 and G08 return one audience, or ask

#	Fix	Evidence	What to change	Passes when
4	**Honest, grounded reasons**	"Hostel life" and "class" invented in G05, G09, G10 and G23; "easy wash" on everything; G23 contradicts itself	In the picker prompt, allow reasons only from the student's words and the item's fields. Check that the headline matches the items. Fix the care tagging (all 300 are "easy")	No reason mentions an occasion the student did not state; headline and items agree in all 30 examples
5	**No-match replies must deliver**	G27 and G28 promise hoodies or tees and show none	For a brand the catalogue does not carry, show the closest in-budget Bewakoof items, labelled as Bewakoof; if there is no budget, ask for one	G27 and G28 show items or ask a question, never a promise
6	**Vague requests and outfits**	G17 mixed joggers, a hoodie and jeans; G01 returned three tees for "3 outfits"	Decide: either say "I pick pieces, not outfits", or build a pair (tee plus bottom) under one total. Use today's season when the student gives no weather	G01 and G17 give a sensible, explained answer

#	Fix	Evidence	What to change	Passes when
7	**Shared-wifi rate limit**	5 requests a minute per IP, kept in memory	Limit per session as well as IP, and keep the count in a shared store	A class of 30 on one wifi can use it
8	**Catalogue depth**	Only 4 black tees under ₹700 (G06) and 4 light cotton shirts (G08); catalogue is 81% men's	Add women's items and cheap basics; fix formality tags	Every happy-path example returns 3 distinct items
9	**Evaluation quality**	Same model judges and generates; no request log; no rules-only baseline run	Use a different judge model, log every request, compare with a cheapest-three-in-budget baseline, and have people rate a sample	Judge agrees with people on at least 8 of 10 samples
10	**Sizing note**	Sizing is the largest cause of returns in the research (figure to verify)	Link the Bewakoof size chart on every card and say plainly that Thread does not predict fit	Every card shows the size-chart link

Not now: the WhatsApp share card (F5), live stock and prices, and virtual try-on.

7. Memory and taste profile design

Build memory in two layers: conversation memory first (it fixes the re-asking), then a taste profile per student. Code does the remembering and merging; the LLM is never trusted to remember.

Layer 1: conversation memory

- The browser keeps the last parsed request and what Thread asked for, and sends them with each message.
- Code merges the new reply into that request. New values replace old ones and everything else carries over, so "1500" after "tee" becomes tee, budget 1500.
- "Cheaper" lowers the budget; "same but black" changes the colour. With no earlier request, Thread still asks, so the old G22 stays valid in a fresh session.

Layer 2: taste profile, one per student

A small JSON object, no name, phone or email. Example:

```
{
  "profile_id": "u_8f3a21",
  "audience": "men",
  "budget": { "typical_max_inr": 1200, "seen": 5 },
  "likes": { "colours": { "black": 4, "olive": 1 }, "fits": { "oversized": 3
}, "patterns": { "solid": 3 }, "categories": { "tee": 4, "hoodie": 1 } },
  "avoids": { "colours": ["yellow"] },
  "uses": { "class": 3, "gym": 1 },
  "bought": [ { "item_id": "BW-555436", "date": "2026-11-12", "rating": 4 }
],
  "updated": "2026-11-12"
}
```

What each signal is worth

Signal	Weight	Example
Bought and rated 4 or 5	Strongest	Taps "I bought this", rates it a week later
Bought and rated 1 or 2	Strong negative	Avoid that fit, fabric or colour
Saved, or chose the top pick	Medium	Taps save on a card
Stated in words	Medium	"I never wear yellow"
Shown but ignored	None	Not counted

Older signals count for less over time, so a changed taste is picked up.

How the profile is used

- Code gives matching items a soft ranking boost, and hides items the student already bought.
- A one-line summary ("usually picks black, oversized tees") goes to the picker so the reason can say why this fits them.

- Hard rules never change: what the student asks for now beats the profile ("not black" wins), and the profile is never a hard filter.
- Start using it only after about 3 signals; before that, assume nothing.
- Never infer body type or skin tone.

Purchases: Thread has no checkout, so students report them. Each card gets an "I bought this" button, and a rating prompt appears about a week later. This is also our north-star measure (bought and rated 4 or more).

Student control: a "What Thread knows about you" panel shows the profile with edit and delete, plus a "Forget me" button.

Where to store it

Option	Pros	Cons
Browser storage, sent with each request (**start here**)	Fits the current app with no database; private by default; fast to build	Lost if cleared; does not follow the student to another device
Anonymous ID plus a hosted key-value store	Survives across visits	Adds a database; data now sits with us
Login accounts	Works on any device	Too heavy for the MVP

8. New test cases and next steps

These 12 cases extend the golden set as **v1** (G31-G42). They are added to the 30 existing examples, which stay unchanged. The CSV `golden_set_v1_additions.csv` goes with this report.

ID	Setup	Input	Must hold
G31	Same session	"tee", then "1500"	Tee recommendations within 1500; no repeated question
G32	Same session	"budget 1000", then "hoodie"	Hoodies within 1000; no repeated question
G33	After a black tee under 700	"cheaper"	Black tees under a lower budget
G34	After an oversized tee under 800	"same but not black"	Oversized tees under 800, no black

ID	Setup	Input	Must hold
G35	Fresh session	"same as before but cheaper"	Says there is no earlier request
G36	Profile of black oversized tees	"something for college, 1000"	Top pick fits their usual taste; not forced
G37	Same profile	"not black, tee under 800"	No black item; the request beats the profile
G38	Profile with a bought item	"plain black tee 700"	The bought item is not shown again
G39	Two different profiles	Same vague request in both	Each result reflects only its own profile
G40	Student taps "Forget me"	"something for college, 1000"	Behaves like a new student
G41	Fresh session	"light cotton shirt, 1000"	One audience, or asks men's / women's / either
G42	Fresh session	"Nike hoodie under 1500"	Shows closest Bewakoof hoodies, or asks; never promises and shows nothing

Order of work

1. Fix speed and the fallback (fix 2) first. A slow app makes every other test unreliable.
2. Add conversation memory (fix 1) and run G31-G35.
3. Fix audience, grounded reasons and no-match replies (fixes 3-5); re-run all 30 original examples plus G41 and G42.
4. Build the taste profile and "I bought this" button (section 7); run G36-G40.
5. Run 5-10 students through the app and record what they bought and rated. This is the only way to measure our north-star metric.

Ship gate for the next version: every original example still passes, all 12 new cases pass, the median recommendation takes under 8 seconds, and there is no generic fallback text. Report results as counts (for example 40 of 42), not percentages.

Open questions for the team

- Do we show pieces only, or build a two-piece outfit under one total? (fix 6)
- Are we comfortable with the script-based catalogue and Bewakoof's terms? (section 2)
- Which real source supports the sizing-returns figure? (section 3)

Sources: Myntra MyFashionGPT, Myntra CEO interview, Amazon Rufus India, Flipkart Flippi, StyleBuddy, app comparison, and our live test of threadstyle.vercel.app on 11 Nov 2026.